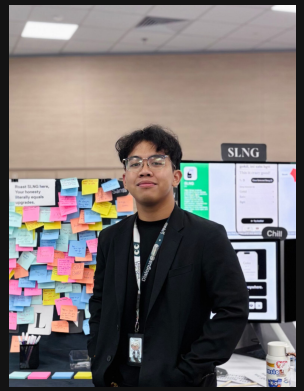
Building **iOS Apps**. Shipping Features. Keeping Them Reliable.

Pramuditha Muhammad Ikhwan

iOS Engineer

Selected Work · 2025–2026



Pramuditha Muhammad Ikhwan

iOS Engineer

An iOS Engineer experienced in shipping production features, strengthening API security, and maintaining app reliability in collaboration with cross-functional teams.

Educational Background

Apple Developer Academy @ Infinite Learning
Learner Cohort 6
Feb 2025 - Dec 2025

Universitas Negeri Yogyakarta
Majoring in Information Technology
Aug 2021 - May 2025

Professional Background

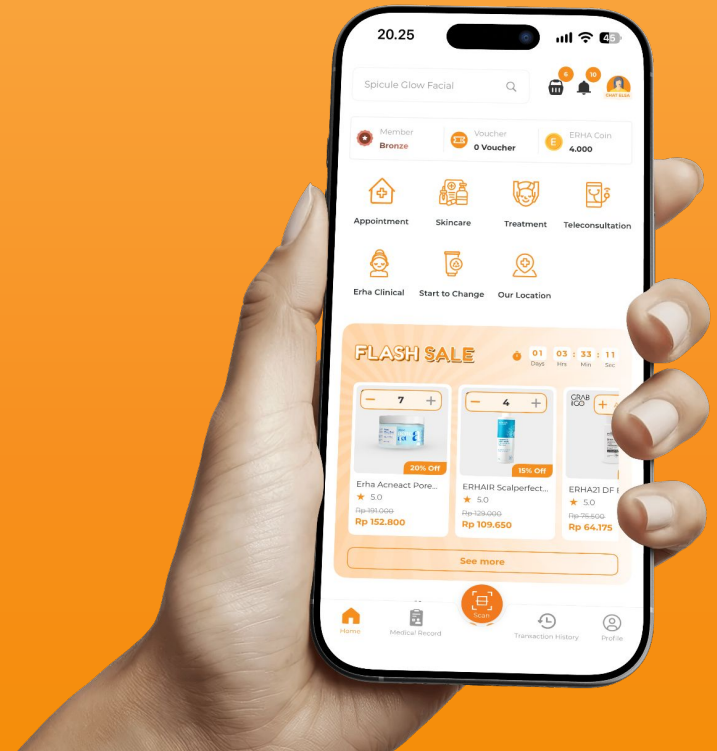
Arya Noble (ERHA Buddy)
iOS Developer
March 2026 - Present

4TITU PTE LTD
Full Stack Developer
Aug 2025 - Feb 2026

PT Widya Inovasi Indonesia
System Analyst
Aug 2023 - Dec 2023

Selected Work in
iOS Engineering.

ERHA Buddy: Evolving and Securing a Production iOS App at Scale



ERHA BUDDY

An app to simplify access to ERHA skincare, beauty, and healthcare services.

Since March 2026, I have worked as an iOS Developer at Arya Noble, developing and maintaining ERHA Buddy within a two-person iOS team. The app serves approximately **40,000 monthly active users**

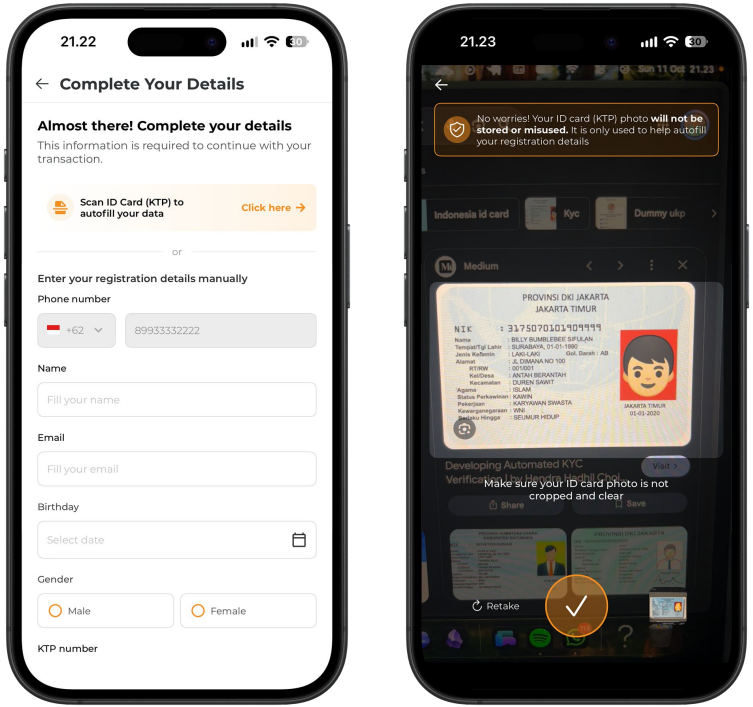
My responsibilities include feature implementation, technical design, sprint planning, estimation, code reviews, release preparation, and post-release monitoring.

Registration Simplification: Letting Users Shop Before Signing In

I reworked the shopping journey so users could browse products and treatments, build a guest cart, select an address, and review their order before authentication became necessary at checkout.

This required more than moving the login screen: I implemented **guest-state persistence, cart and address synchronization after login**, context-aware authentication gates, and post-login revalidation of prices and promotions.

I also simplified registration through centralized validation, normalized phone-number handling, and optional KTP scanning and autofill. Post-release analytics showed a **98.1% quarter-over-quarter increase in average daily cart checkout intent**, an outcome the revised journey contributed to.



Flash Sale: Implementing the iOS Purchase Journey

I implemented the iOS Flash Sale journey across Home, campaign catalogs, product and treatment details, cart, order summary, checkout, and transaction details.

The implementation **covered Internal and External Flash Sale flows**, including their different cart and fulfillment requirements. I handled campaign countdowns, promotional pricing, stock-aware quantity limits, clinic selection, and campaign availability changes. A substantial part of the work involved keeping these rules consistent across screens, especially **when a campaign changed state or the server adjusted availability** during an active shopping session.

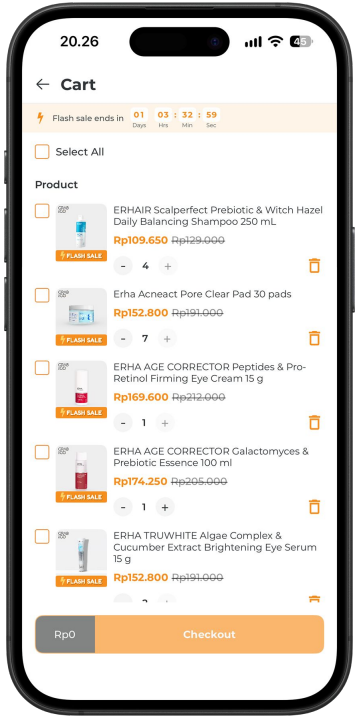


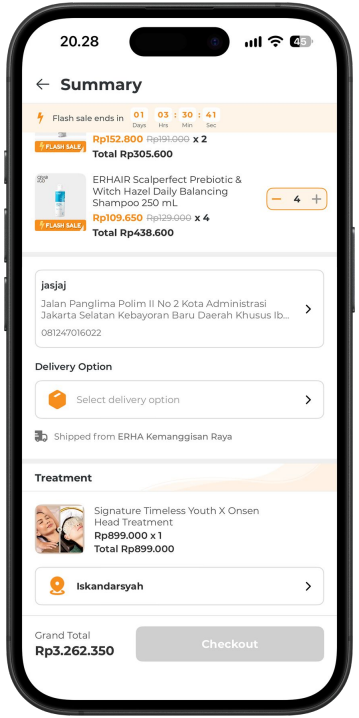
SSE: Keeping Flash Sale Screens Updated

I implemented an application-wide Server-Sent Events connection to receive Flash Sale lifecycle updates and refresh affected screens.

The client handles incremental **event parsing, heartbeats, retry instructions, network recovery, authentication changes, and foreground/background transitions**. I also added reconnect behavior and a local deadline fallback for upcoming campaigns. Events trigger REST refreshes, while REST responses remain authoritative for prices, stock, eligibility, and checkout state. Refresh coordination reduces overlapping requests and keeps Home, catalogs, product details, cart, and order summary aligned as campaign conditions change.

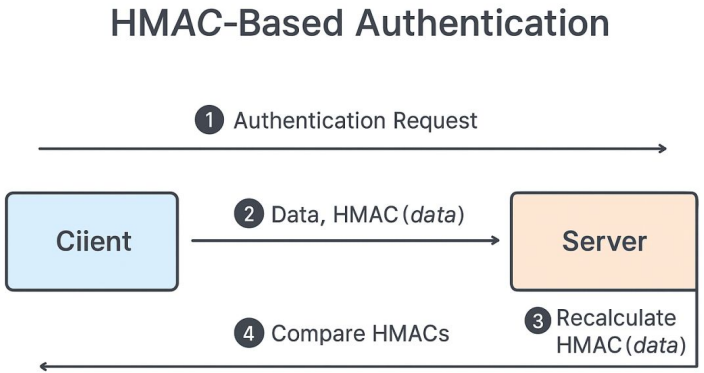
Complete example implementation:
https://medium.com/@prammmoe/getting-to-know-server-sent-events-and-actually-implementing-it-on-ios-a5cd22b1daf8





HMAC: Making the Client and Server Agree on Each Request

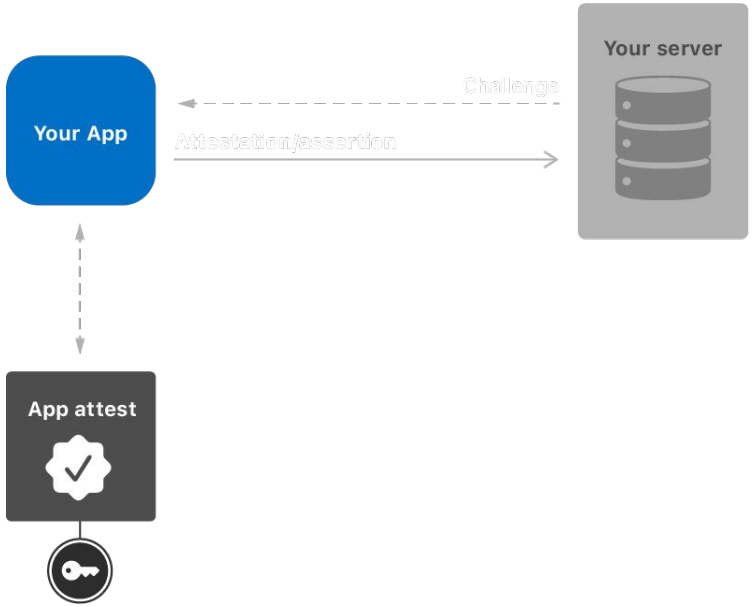
I designed and implemented HMAC request signing on the iOS side, aligned with the backend verification contract. Signing covers **canonical request paths, query parameters, payloads, nonces, and timestamps**, with the required signature headers attached through the shared networking layer. Centralizing this behavior made request protection consistent across API callers and reduced duplicated signing logic. The work also required handling URL encoding and serialized payloads correctly, so the client and server could verify the same request representation, and restricting internal integrity headers when requests reached external hosts.



Complete implementation:
https://prammmoe.space/blog/securing-api-requests-with-hmac-in-swift/

App Attest: Handling What Happens When Verification Fails

I implemented the iOS Apple App Attest integration, including backend challenge retrieval, device-key registration, Keychain persistence, and per-request assertions. I handled **failure paths** such as **invalid or unknown keys**, **failed native assertions**, and **backend-rejected proofs**, with bounded recovery and retry behavior. Registration is coordinated so concurrent requests do not independently create competing keys. The rollout also accounted for app-version compatibility and simulator limitations. Recovery distinguishes safe requests from mutating operations, avoiding automatic repetition of business actions when the server provides no idempotency guarantee.



Calculate Order: Removing Redundant Requests

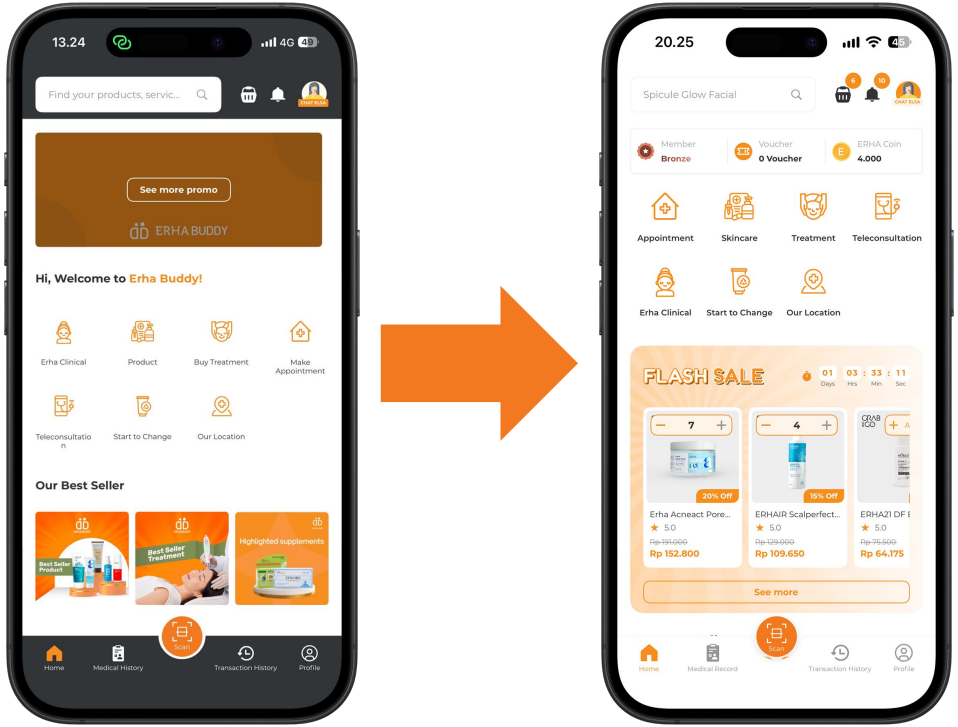
I optimized the Calculate Order flow by changing when calculations are triggered and coordinating requests with equivalent inputs. The summary can display preliminary fees before a payment method is selected, reuse a valid quote, or join an equivalent request already in progress. Changes that affect pricing invalidate the relevant quote, while stale responses cannot overwrite newer totals. In the observed calculation flows, API calls decreased from 7–9 to 1–3. The largest reduction was approximately **89%, reducing repeated calculation work** and the associated backend processing overhead.

Checkout Revamp: Submitting Server-Issued Quotes

I revamped checkout across combined product and treatment orders, teleconsultation, and Flash Sale using a server-issued **`quoteId`**. The client submits this identifier for checkout, reducing reliance on client-supplied pricing values. I integrated quote handling with payment methods, vouchers, ERHA Coins, delivery and pickup selections, and stock adjustments. The implementation also handles quote invalidation, expiry, and checkout conflicts, so changed order inputs require a fresh calculation before submission. This moves more pricing authority to the backend while keeping the order summary consistent with the quote being purchased.

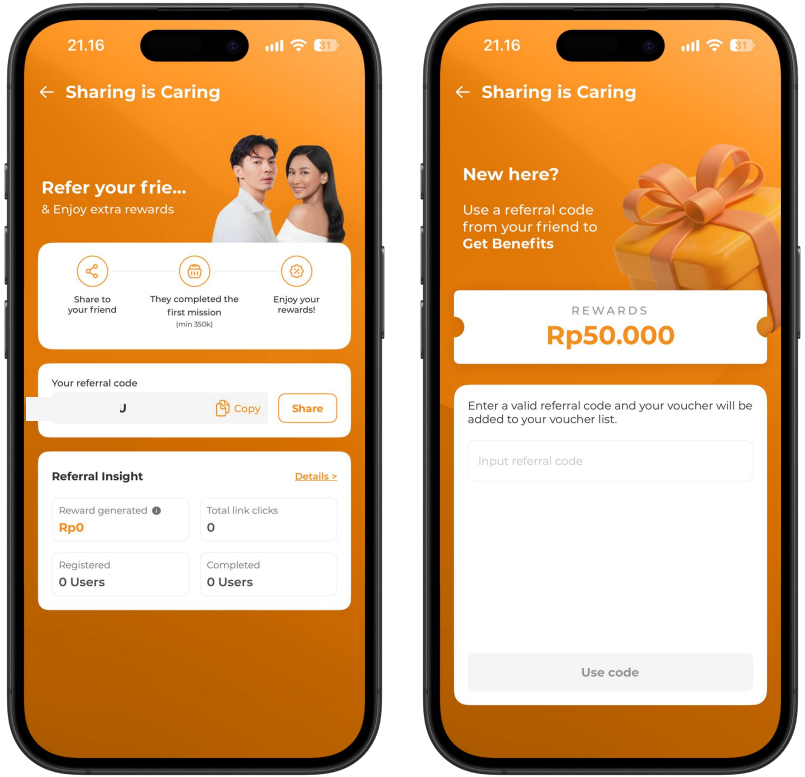
Home Revamp: Implementing the Updated Screen

I implemented the updated Home screen design across the search area, menu grid, membership information, voucher and ERHA Coins sections, Best Seller banners, Flash Sale entry points, and bottom navigation. The work included layout and asset updates, API-response mapping, and loading, empty, and error states. **I also resolved differences between guest and authenticated layouts**, clipped menu labels, inconsistent banner presentation, and sections that appeared incorrectly when their data was unavailable. My contribution focused on **translating the supplied design into the existing app structure** and handling the states beyond the populated design mockup.



Referral: Translating the Design into iOS Screens

I implemented the Referral interface and its navigation within the existing membership and account flows. This included the referral guide, personal-code display, copy and share actions, referral-code input, and referred-user lists with Registered and Success states. I added keyboard handling, scrollable layouts, loading skeletons, pull-to-refresh, and state-dependent feedback. My scope covered **UI implementation and integration with the existing referral behavior**. I also corrected the date used for referral transaction display and sorting to reflect the user's registration time.



Building SLNG: My Journey to Craft a **Fast, Native, Playful App**



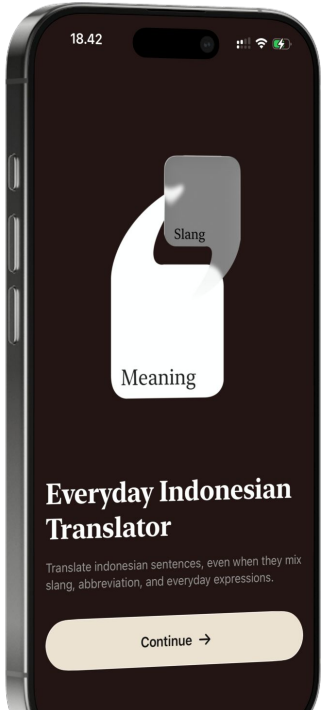
SLNG

An app to help foreign students understand informal Indonesian

Foreign students struggle to understand the meaning and social context of Indonesian slang and slang evolves quickly, which makes it hard for them to keep up.

From a developer’s perspective, the interesting problem wasn’t just the slang itself—it was how to design an iOS experience that handles unpredictable, fast-moving language gracefully.



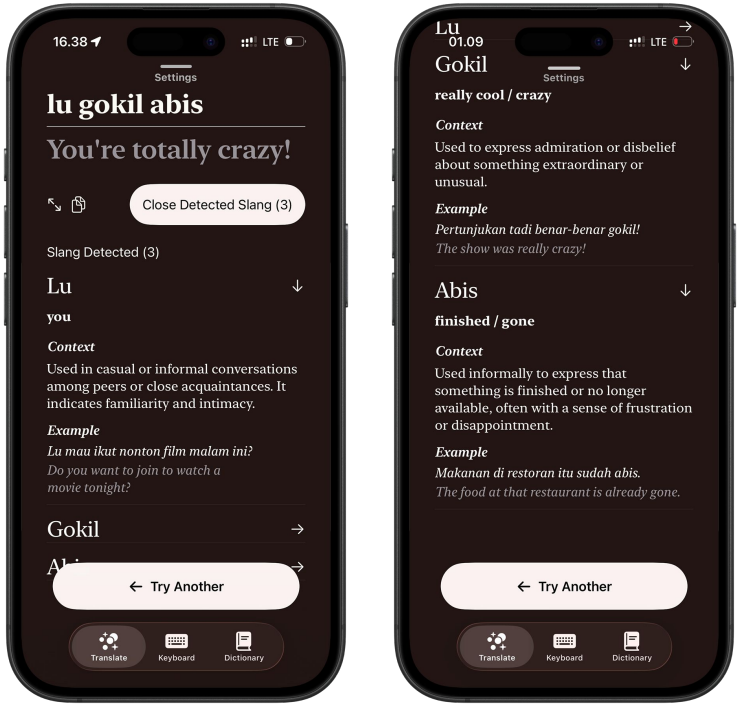


PROBLEM STATEMENT

Foreign students struggle to grasp meaning + social context, not just words.

Many foreign students could recognize Indonesian slang, but not the intention behind it. Meaning shifts with tone, sentiment, and social context. One word can change entirely depending on how it's used.

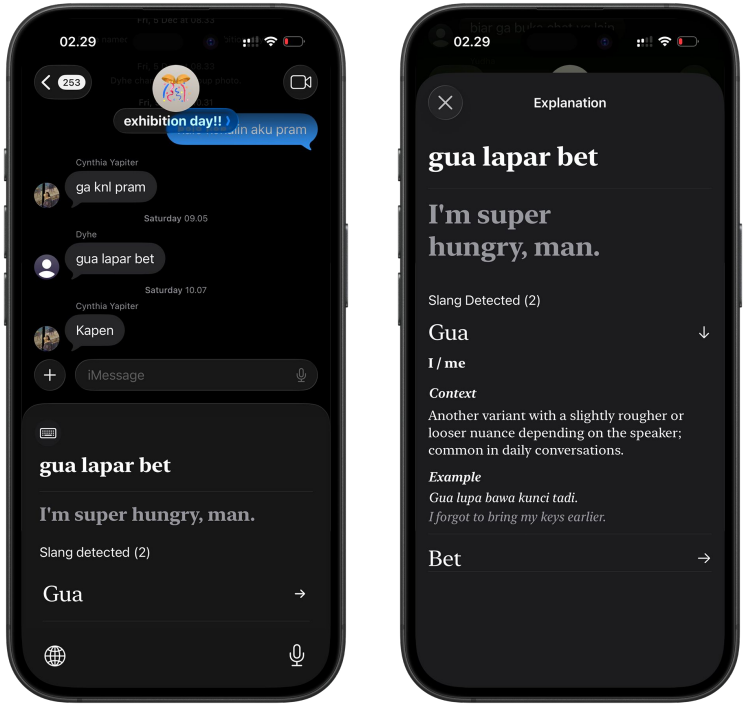
I built SLNG's data model to capture that nuance. Each slang has contextual variants, sentiment labels, and bilingual examples, paired with a matching algorithm that handles spelling variations and tone alignment. This lets SLNG detect not just the slang, but the meaning in that moment, making translations far more intuitive.



Designing a translator that captures nuance, not just definitions.

Instead of adding more features, I focused on what actually helps users interpret meaning in real conversations.

SLNG is built around multiple entry points, including the main app, a custom keyboard, and a system-wide Share Extension, ensuring slang can be understood wherever it appears.



Avoiding a flat UX by playing with motion, sounds, and haptics.

SLNG needed a personality, something more expressive than a static dictionary UI. I shaped the experience with motion, sound, and haptics to match the lively nature of slang.

Interactions use fluid spring animations, subtle haptic cues, and light audio touches that make actions feel tactile without becoming gimmicky. Everything follows iOS-native principles: responsive, intentional, and lightweight. The goal wasn't visual noise, but an experience that feels expressive and genuinely fun to use.



Each round of testing reshaped flow, interactions, and stability.

SLNG evolved through fast iterations shaped by real user behavior. While testers noted minor inconsistencies in gestures and transitions, a key insight came from foreign students who often heard slang without knowing how to spell it. This led to the addition of Speech-to-Text and a refined input flow for real-world voice conversations.



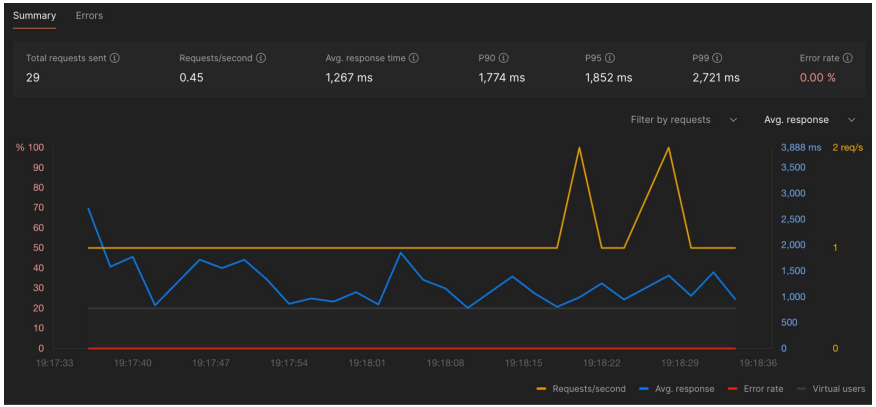
Iterating Based on Real Feedback

Migrating to Go to halve response times and stabilize concurrency.

As usage scaled, backend latency became a clear bottleneck. The initial Node.js prototype worked for early validation, but it struggled with consistent response times under concurrency.

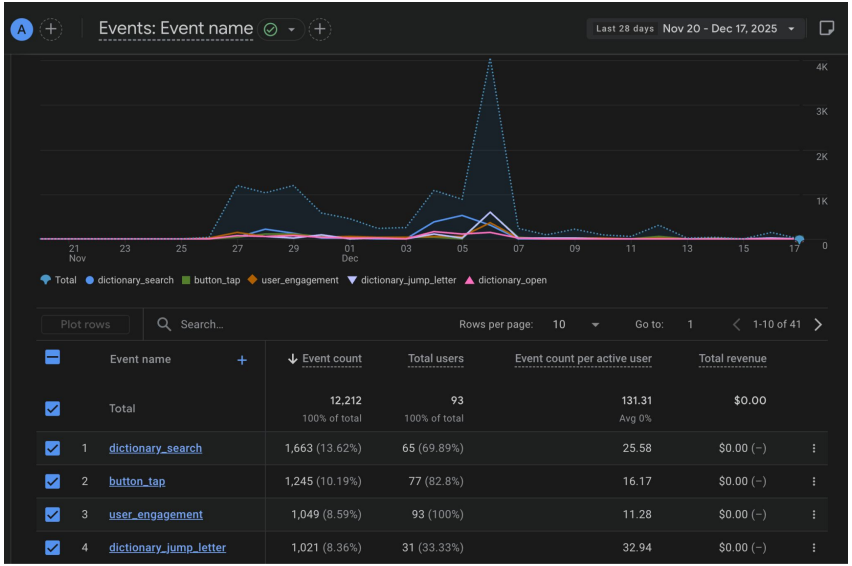
Rather than patching around the problem, I rebuilt the backend in Go. The migration enabled lightweight routing, stronger concurrency handling, and more efficient memory usage. Average response times dropped from ~3 seconds to ~1.5 seconds, effectively cutting latency in half.

This wasn't just an engineering improvement. The faster backend translated directly into a more fluid, responsive, and reliable user experience.



Tracing Real User Behavior with Firebase Analytics

To inform future product decisions, I used Firebase Analytics to trace which features users actively engaged with. Feature-level usage data revealed where users found real value, allowing me to prioritize iterations and focus development effort on areas that mattered most.



Tech Stack of **SLNG**



Swift Data



AVFoundation

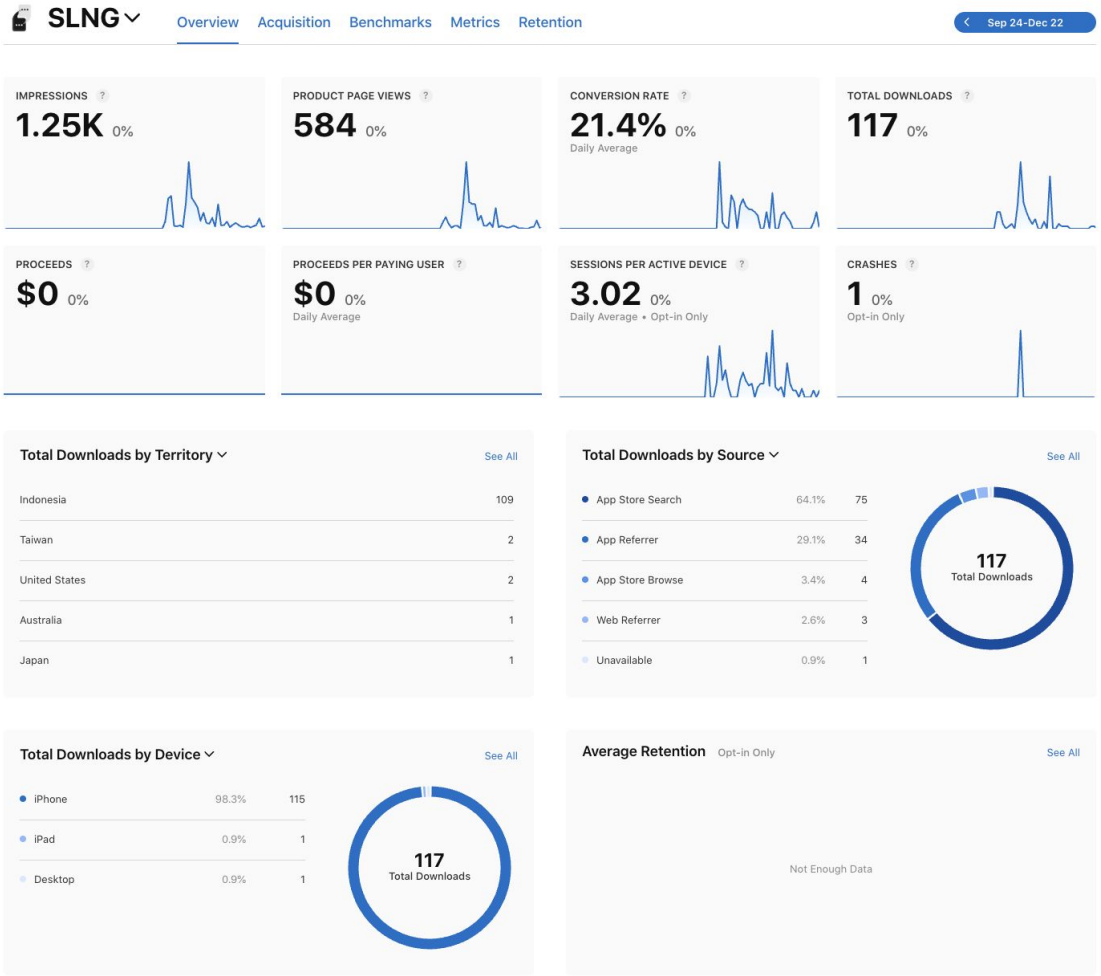


OpenAI API



Apple Speech

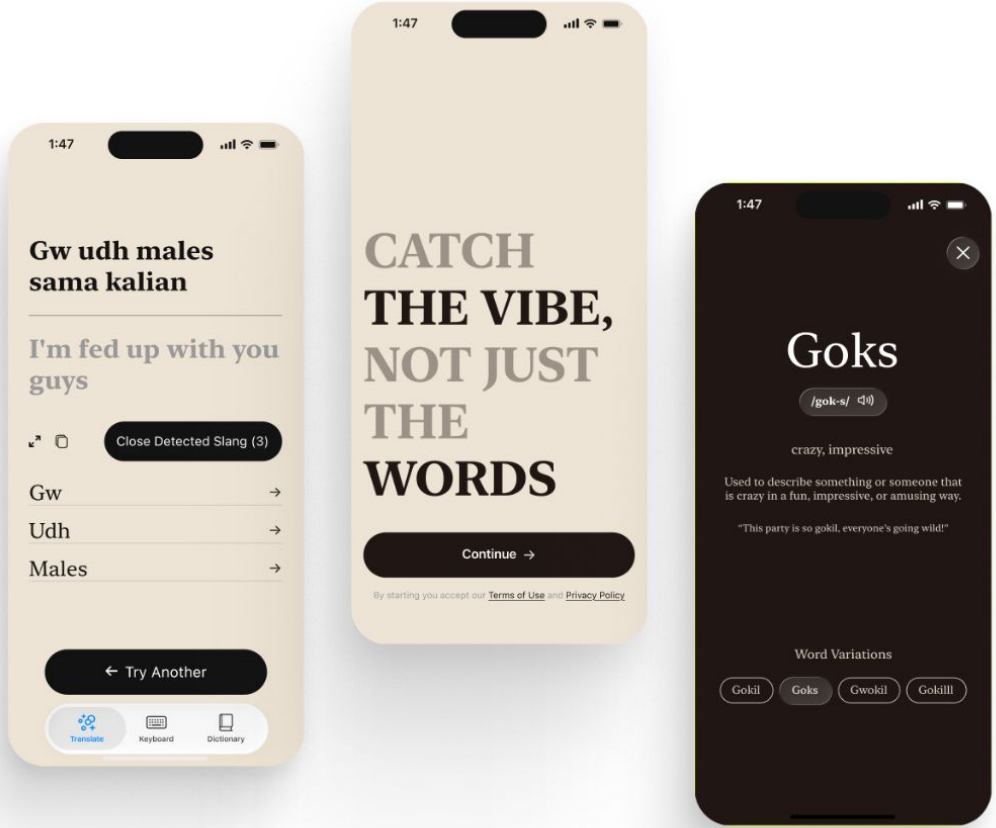
Metrics of **SLNG in App Store**



SLNG

Indonesia Slang Translator

Download on the App Store



Building Paintee: Crafting Face-Aware AR Experiences from First Principles



Paintee

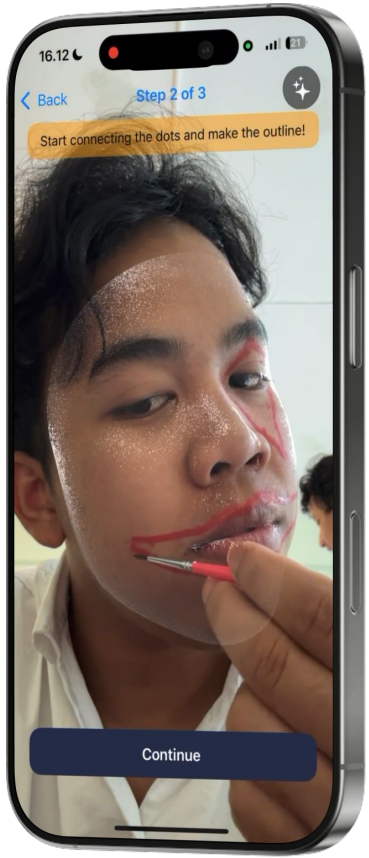
An app to help beginner doing face painting who find it difficult to create proportional and symmetrical face painting

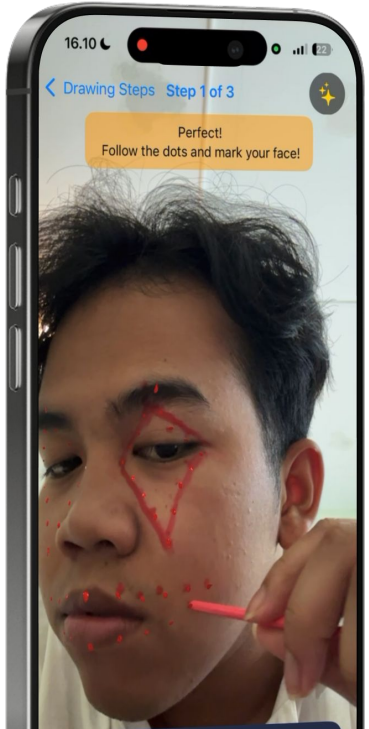


Beginner who apply face painting on their own faces often struggle with proportion and symmetry, especially when working from references on a flat surface.

Looking at it through a builder’s lens, the interesting problem wasn’t painting itself—it was how to design an experience that understands facial structure and guides precision in real time.

PROBLEM STATEMENT

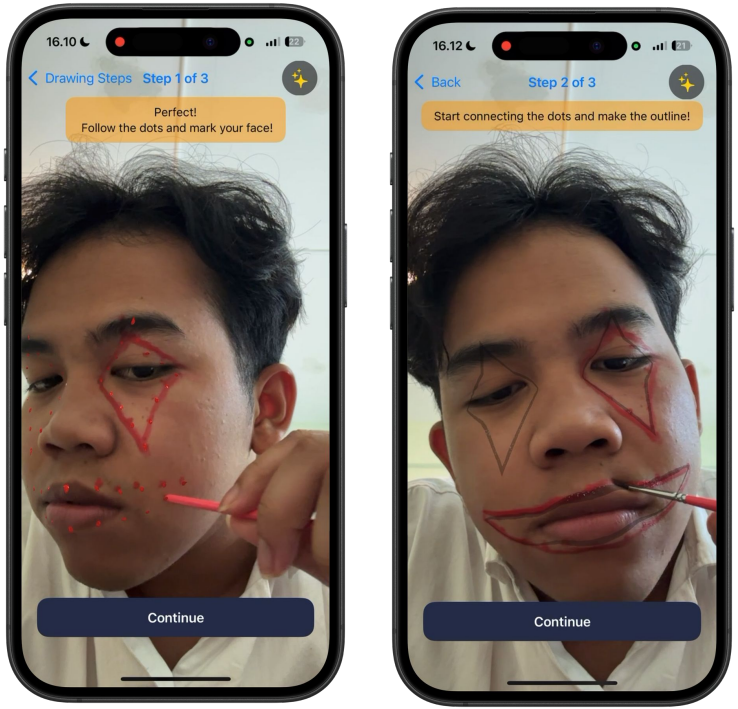




Crafting Navigation Around a Continuous AR Session

Face painting is a continuous, hands-on activity, not a sequence of screens. Every interruption breaks focus, muscle memory, and spatial alignment.

In Paintee, navigation had to respect the session itself. Instead of resetting context at every step, the app preserves a single, continuous AR session while guiding users through structured phases—previewing, dotting, outlining, and painting. Screens change, but the session doesn't. This approach turns navigation into session control, allowing beginners to stay grounded in the same physical space while progressing with confidence and precision.



Voice Control as a First-Class Interaction

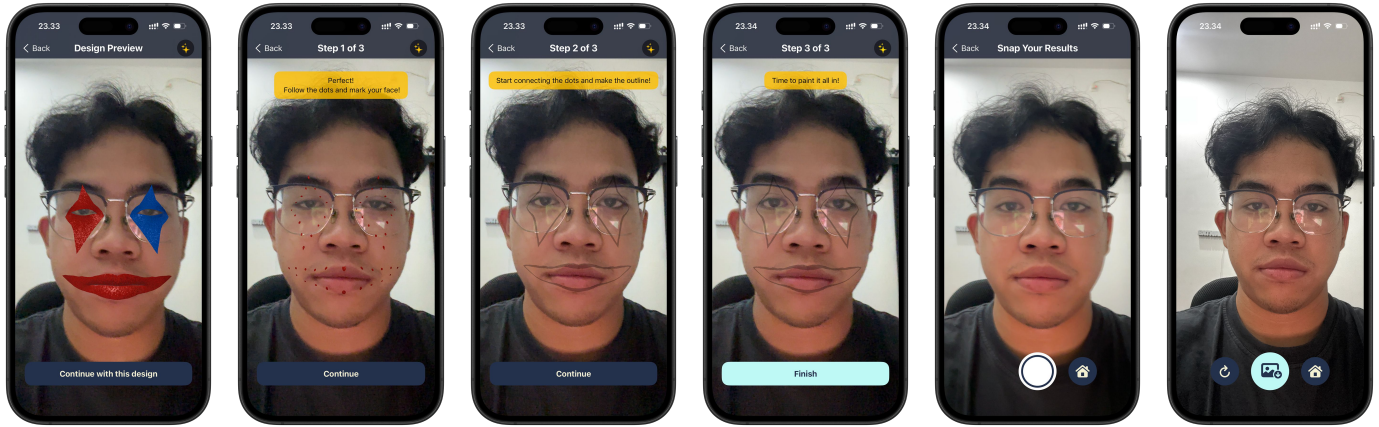
During face painting, hands are already busy and attention is split between a mirror, the brush, and facial precision. Asking users to constantly tap buttons or navigate UI breaks focus at the exact moment it matters most.

By allowing users to move between steps, toggle guidance, and progress through the experience using voice, the app removes unnecessary touch interactions. This lets users stay focused on applying the paint itself, keeping their hands, posture, and attention anchored to the task instead of the screen.



Nothing Extra Between the User and the Task

Paintee is structured as a single, linear session rather than a collection of screens. From design preview to guided application and finally capturing the result, each step advances the same underlying state instead of creating a new context. The AR session stays alive while the UI layers shift just enough to guide the next action. This keeps navigation lightweight and predictable, avoids unnecessary resets, and preserves spatial alignment throughout the process. By treating the experience as one continuous flow, the app removes friction from the interface and lets users stay focused on the physical act of face painting.



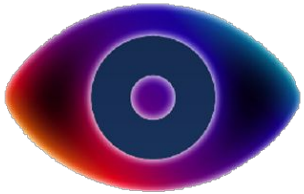
Tech Stack of **Paintee**



RealityKit



SwiftUI



Vision



ARKit

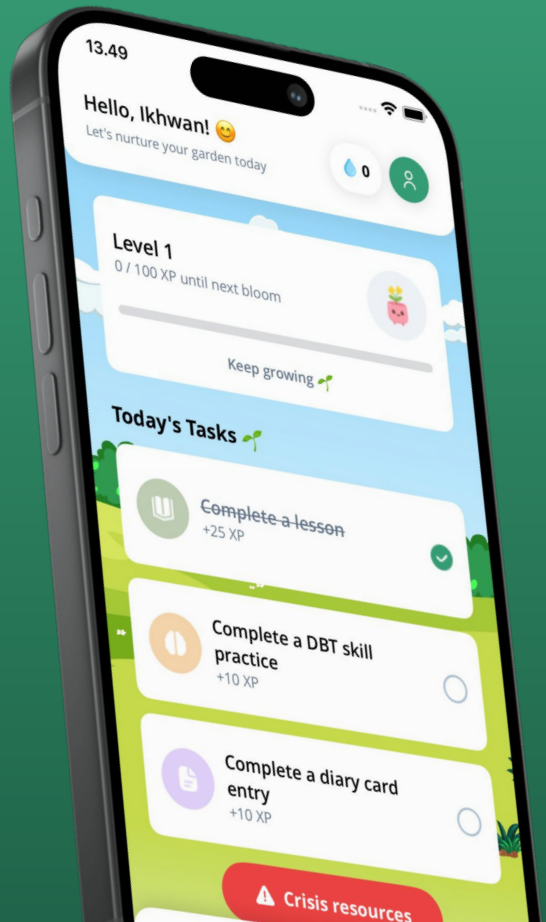
PAINTEE

Now available on **App Store!**

Download on the App Store



Contributing to Rooted DBT: Developing DBT Learning Flows



Rooted DBT

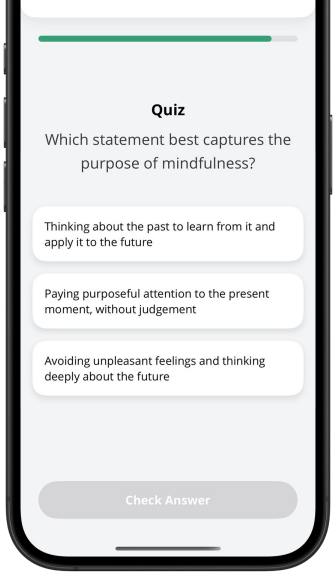
An app to make DBT skills made simple, daily, doable.

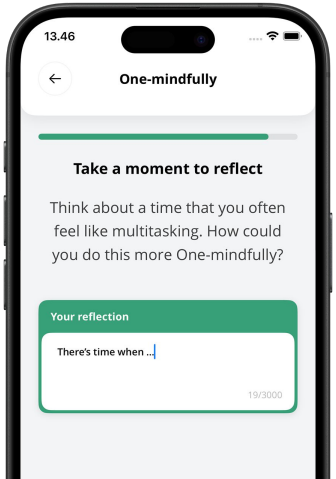
Implementing Clean Routing for Dynamic Content

In the Mindfulness lesson module, I implemented a clean routing structure to support highly dynamic lesson content without coupling navigation logic to individual views.

To handle the variability of mindfulness lessons, I focused on separating routing logic from presentation. Navigation between lesson steps was driven by a centralized routing layer instead of view-specific state changes. This allowed lessons to progress linearly or conditionally without duplicating navigation code across screens, resulting in a cleaner mental model and easier maintenance as lesson structures evolved.







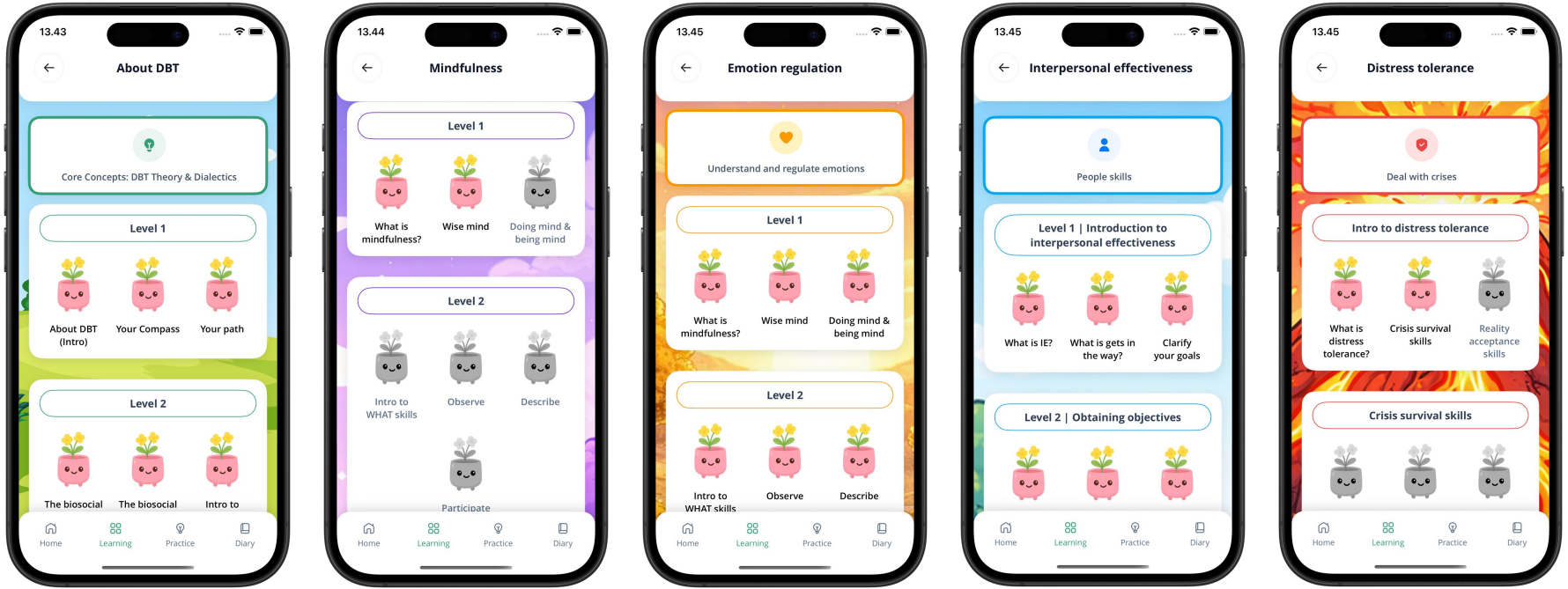
Leveraging Generics for Flexible Lesson Content

I designed lesson sections and items as generic data structures constrained by **`LessonContent`** and **`LessonRoute`** protocols. This made it possible to reuse the same **`LessonItemCellView`** across multiple lesson configurations, while keeping routing explicit and type-safe. As a result, new lesson variations could be introduced by defining new content and routes, without modifying the core component or navigation flow.

```
LessonItemCellView<MindfulnessLesson, MindfulnessLessonRoute>
(   level: level.level,
    title: nil,
    items: level.items,
    levelHeaderStrokeColor: Color(hex: "#7030E0"),
    onTap: { item in
        guard !item.isLocked else { return }
        router.navigate(to: item.route)
    }
)
```

```
LessonItemCellView<DistressToleranceLesson, DistressLessonRoute>
(   level: nil,
    title: level.title,
    items: level.items,
    levelHeaderStrokeColor: Color.red500,
    onTap: { item in
        guard !item.isLocked else { return }
        router.navigate(to: item.route)
    }
)
```

With the help of the generics, these views can be implemented with just a minimum line of code.



Key Learnings

This project deepened my understanding of how engineering decisions directly affect user focus in mindfulness experiences. Clean routing, reusable components, and minimal UI friction were not just technical choices, but product constraints. I learned to balance flexibility with simplicity, ensuring the system could grow without exposing complexity to the user. Contributing through small, well-scoped changes also reinforced the value of iteration over large, disruptive rewrites in a shared codebase.



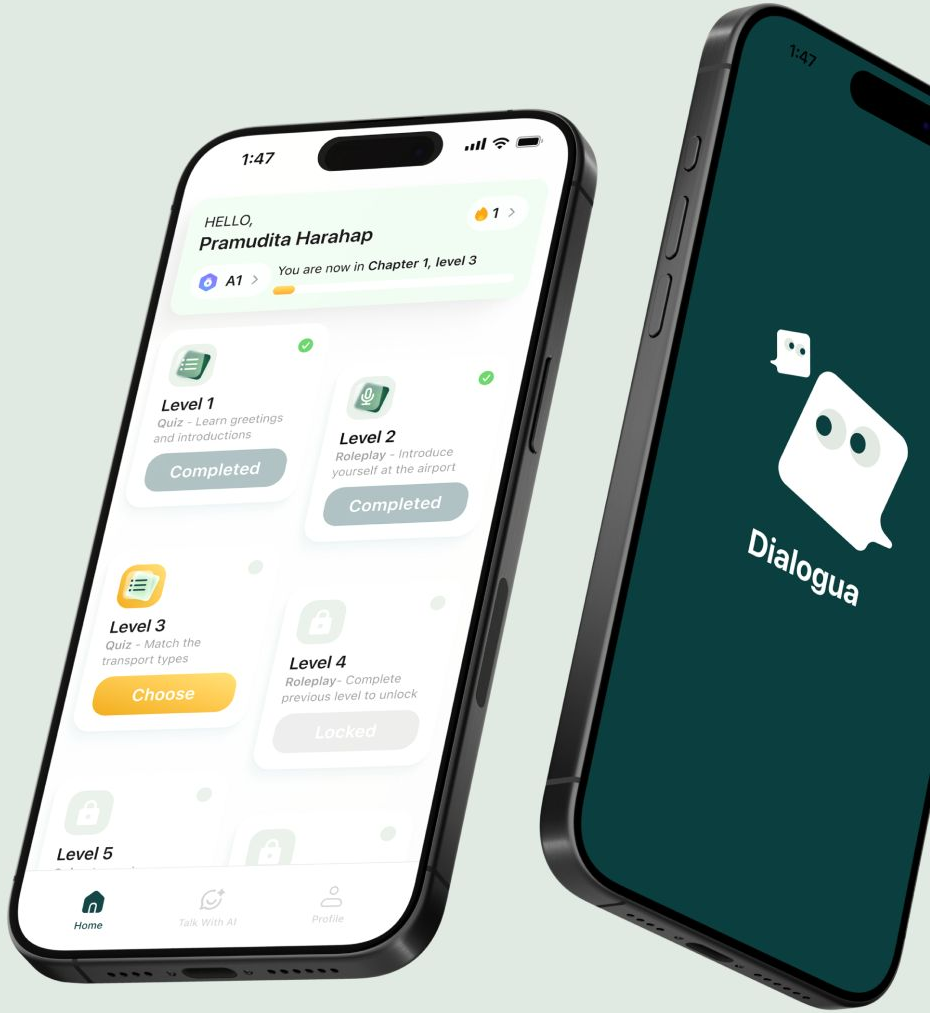
Beyond the **Case Studies.**

Dialogua

An app to help foreign students in Indonesia reduce language anxiety by providing step-by-step speaking practice in realistic social scenarios, so they can build confidence, progress from simple to complex interactions, and track their improvement.

Role: iOS Developer

Try It Now





Miruu

An app to turn your daily life into an RPG adventure, helping you build habits, grow skills, and track personal progress through quests, XP, and evolving characters.

Role: iOS Developer





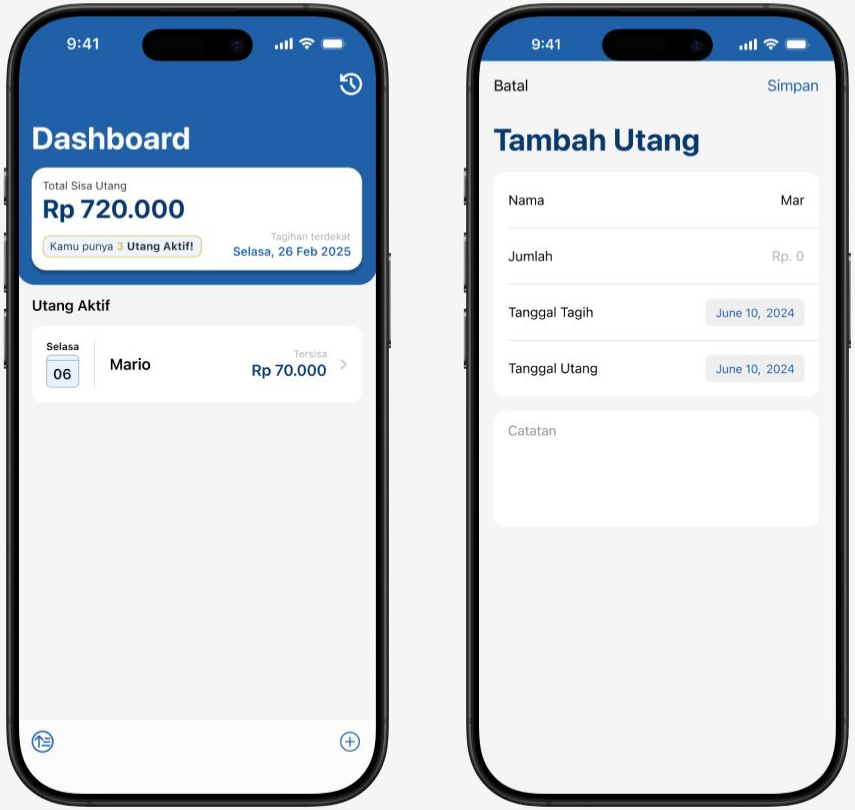




FreeUtang

An app to manage and track your receivables

Role: iOS Developer



Built by curiosity. Proven by shipping.

Open to iOS developer and engineering roles

linkedin.com/in/ikhwanpramuditha/

github.com/prammmoe

+62 812 4701 6022

medium.com/@prammmoe

